

Journal of Supercomputing, 14, 5–17 (1999)
© 1999 Kluwer Academic Publishers. Manufactured in The Netherlands.

An Efficient General In-Place Parallel Sorting Scheme

S. Q. ZHENG †, BALAJI CALIDAS ‡ AND YANJUN ZHANG §

† *Department of Computer Science, University of Texas at Dallas, Richardson, TX 75083, USA*
‡ *Department of Electrical and Computer Engineering, Louisiana State University, Baton Rouge, LA 70803, USA*
§ *Department of Computer Science and Engineering, Southern Methodist University, Dallas, TX 75275, USA*

Abstract. We present a simple and general parallel sorting scheme, ZZ-sort, which can be used to derive a class of efficient in-place sorting algorithms on realistic parallel machine models. We prove a tight bound for the worst case performance of ZZ-sort. We also demonstrate the average performance of ZZ-sort by experimental results obtained on a MasPar parallel computer. Our experiments indicate that ZZ-sort can be incorporated into a distributed memory parallel computer system as a standard routine, and this routine is useful for space critical situations. Finally, we show that ZZ-sort can be used to convert a non-adaptive parallel sorting algorithm into an in-place and adaptive one by considering the problem of sorting an arbitrarily large input on fixed-size reconfigurable meshes.

Keywords: parallel algorithm, parallel architecture, performance evaluation, scalability, sorting, supercomputing

1. Introduction

According to current and projected future technologies, it is not unusual that hundreds of thousands and even more processors are employed to collectively solve a problem. Such massively parallel computer architectures are justified because of the demands for enormous computing power that arise in many important scientific, engineering, information and knowledge processing applications. These applications are characterized by their very large problem sizes.

In a distributed memory massively parallel computer, each processor is associated with a local memory, and a processor/memory pair forms a node. Data communication and process synchronization among nodes are achieved by message passing through an interconnection network. Ignoring memory management issues by simply assuming that memories are inexpensive and the users can use as much internal memory space as they want may not be realistic for many applications. In fact, the space efficiency is more important and critical in parallel computing than that in sequential computing. Many applications of parallel computers are memory-bounded rather than CPU-bounded or I/O bounded. For fixed amount of memory, the larger the number of processors in a system, the less local memory for each processor. Due to this restriction each node can only be used to solve a small subproblem. Operating in a multi-user environment, a parallel computer system is partitioned

into several virtual machines with different hardware allocations for multiple users. Since user memory spaces may be disjoint, the limited local memory capacity restricts applications of all users.

In parallel computing, space efficiency is related to scalability. Amdahl's law states that with the same problem size (or, in other words, the same work load), the speedup of a parallel computing system over a sequential computing system is limited by a constant regardless of how many processors are employed [4]. This pessimistic view is based on a fixed inherently sequential fraction of the solution. Indeed, for a fixed computing load, the system efficiency decreases drastically as the number of processors used increases. Gustafson considered using parallel computers to solve scaled-up problems [6]. He showed that under the same execution time constraint, system efficiency does not drop sharply when using a scaled-up parallel computer to solve a scaled-up problem. Gustafson's analysis of achieving best system efficiency by scaling-up problem under the fixed-time assumption is commonly referred to as Gustafson's law [7]. Sun and Ni analyzed the speedup and efficiency of an algorithm/architecture combination under the assumption that a problem is scaled-up by using up all available memory space [13]. They showed that for some problems, it is possible to achieve the same or even better efficiency using this approach. In fact, they showed that, under certain conditions, both Amdahl's law and Gustafson's law are special cases of their fixed-memory model. In general, given an algorithm A on a particular architecture C for solving problem P, A is scalable with respect to C if A can maintain efficiency when the size of P increases. That is, when A is used to solve P of larger size by using available memory of C, the sequential fraction and communication overhead of A do not increase significantly compared with solving P of smaller size using A on C. We say that a parallel computer system C is scalable for an application domain if most algorithms in that domain are scalable as the number of processors in C increases. Truly scalable computing systems are hard to build due to physical constraints. However, for a given problem P and a particular machine C, one can design an algorithm which explores the parallelism existing in P and the memory available in C to achieve good algorithm scalability.

In this paper, we consider the design and implementation of in-place parallel sorting algorithms on realistic machine models. Sorting is one of the most fundamental computational problems. Many applications consist of sorting as a subproblem. Therefore, it is important to develop a scalable parallel sorting algorithm that can be implemented on many different computing platforms. An in-place parallel sorting algorithm is one that requires $N + O(p)$ memory space, where N is the number of elements to be sorted and p is the number of processors used. An algorithm may perform poorly due to frequent I/O transfers caused by the limitation of internal memory space. Therefore, in-place parallel sorting may be desirable. In [5], an in-place parallel sorting algorithm based on a shared-memory abstract machine model, $PRAM$, is presented. This algorithm is not suitable for our purpose. In a project reported in [3], three sorting algorithms, bitonic sort, radix sort and sample sort, were implemented on the Connection Machine CM-2 and their performances were compared. Among these three algorithms, only bitonic sort is in-place. The time complexity of its implementation is $O(\frac{N}{p}\log^2 N)$. In-place bitonic merge operations are suitable for the CM-2 machine, since the processors in the CM-2 machine

are connected by a hypercube interconnection. For bounded-degree interconnection networks, such as the one used in the MasPar MP-1 machine, bitonic sort cannot be efficiently implemented.

We present a simple and general parallel sorting scheme, ZZ-sort, that can be used to derive a class of efficient in-place sorting algorithms on realistic parallel machine models. The ZZ-sort uses a sorting procedure on a p-node machine C as a subalgorithm. This procedure sorts p elements, one from each node, in $O(f(p))$ time. We present both theoretical and experimental analysis of ZZ-sort. We prove a tight bound for the worst case time performance of ZZ-sort. More specifically, we show that ZZ-sort sorts N, $N \geq p$, elements on C in $O(\frac{N}{p} \log p(f(p) + \log \frac{N}{p}))$ time, and there exist problem instances that require $\Omega(\frac{N}{p} \log p(f(p) + \log \frac{N}{p}))$ time. We also discuss the performance of an implementation of ZZ-sort on a MasPar MP-1 parallel computer of 8192 processors. Our results indicate that ZZ-sort is quite scalable. It can be incorporated into a distributed memory parallel computer system as a standard routine, and this routine is useful for space critical situations. Finally, we show that our ZZ-sort can be used to convert a non-adaptive parallel sorting algorithm into an in-place, adaptive one by considering the problem of sorting an arbitrarily large input on fixed-size reconfigurable meshes.

2. ZZ-Sort and Its Analysis

Given a distributed memory parallel machine of p processors P_i, $0 \leq i \leq p-1$, we maintain a *queue* Q_i in the local memory of processor P_i as shown in Figure 1. The interconnection network used can be a multi-stage dynamic network, a direct (static) network, or a reconfigurable network such as reconfigurable multi-ring [1] and reconfigurable mesh [12]. The elements of an input set are distributed evenly, but arbitrarily, in the p queues such that each queue has at most $\lceil \frac{N}{p} \rceil$ elements, where N is the number of elements of the input. We say that the queues are *globally* sorted if $x \leq y$ for any x in the ith queue and y in the jth queue where $i < j$.

The ZZ-sort scheme consists of two major phases, the *global phase* and *local phase*. In the global phase, the queues are globally sorted. In the local phase, each queue is sorted sequentially by its dedicated processor, and all queues are concatenated as the sorted output. The global phase proceeds in iterations; each consists of two steps. In the first (resp. second) step of an iteration, each processor P_i such that i is even (resp. odd) extracts the largest (resp. smallest) element of Q_i and puts it in register R_i of P_i. These p elements are sorted by a given parallel sorting algorithm SORT in the ascending order, one element per R_i. Then, each processor P_i inserts the element in R_i into Q_i. The two complementary "zigzaging" element selection rules will guarantee that all elements to be globally sorted within a finite number of iterations, and once sorted, no changes are made to the set of elements in each queue by additional iterations. We use ZZ-sort to stand for "zigzag sort"; its details are given below.

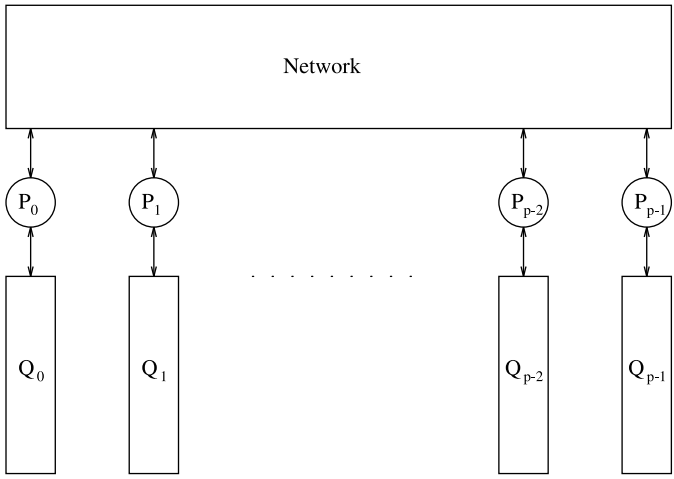


Figure 1. High-level implementation of ZZ-sort.

Algorithm ZZ-sort
begin
/* Initialization */
for all P_i, $0 \leq i \leq p-1$ **do in parallel**
$Q_i \leftarrow$ at most $\lceil \frac{N}{p} \rceil$ elements of an input of size N
endfor
/* Global Phase */
repeat until the queues are globally sorted
/* Step 1 */
for all P_i, $0 \leq i \leq p-1$ **do in parallel**
if i is even then $R_i \leftarrow deletemax(Q_i)$;
else $R_i \leftarrow deletemin(Q_i)$;
endfor
Use algorithm *SORT* to sort elements in R_i's;
for all P_i, $0 \leq i \leq p-1$ **do in parallel**
$insert(Q_i, R_i)$;
endfor
/* Step 2 */
for all P_i, $0 \leq i \leq p-1$ **do in parallel**
if i is even then $R_i \leftarrow deletemin(Q_i)$;
else $R_i \leftarrow deletemax(Q_i)$;
endfor
Use algorithm *SORT* to sort elements in R_i's;
for all P_i, $0 \leq i \leq p-1$ **do in parallel**
$insert(Q_i, R_i)$;
endfor

endrepeat
/* Local Phase */
for all P_i, $i = 0, 1, \ldots, p-1$ **do in parallel**
Sort elements in Q_i sequentially, and output its sorted sequence.
endfor
end

In ZZ-sort, *deletemin*(Q_i) and *deletemax*(Q_i) are the operations that delete the minimum and the maximum from Q_i, respectively, and return the deleted element, and *insert*(Q_i, R_i) is the operation that inserts the element in R_i into Q_i. If $N < p$, a special element is added to each empty queue. The added elements are removed afterwards. The global phase is terminated either after a provable number of iterations that guarantees the queues are globally sorted, or by detecting the termination condition as follows. Let M_i and m_i denote the maximum and minimum of Q_i, respectively, and assume p is even. The queues are globally sorted if $M_0 \le m_1, M_1 \le m_2, \ldots, M_{p-2} \le m_{p-1}$. We may introduce a termination test at the end of each iteration as follows. The M_i element of each P_i such that i is even is sent to P_{i+1}. Then, all P_j's with odd j compare its m_j with the M_{j-1} it received. If $m_j \ge M_{j-1}$, set its flag $done_j$ to be 0, else set $done_j$ to be 1. For all P_i with even i, set $done_i$ to be 0. Clearly, all queues are globally sorted if and only if the sum of all $done_i$'s, $0 \le i \le p-1$, is 0.

We show that ZZ-sort globally sorts p queues within $\lceil \frac{N}{p} \rceil (\lceil \log_{\frac{4}{3}} p \rceil + 1) + 1$ iterations on any input of size N. We also show that this upper bound is tight within a constant factor by constructing an input instance for which $\frac{N}{3p} \log_2 p$ iterations are needed. We use the "zero-one principle" (Lemma 2) to reduce the sorting problem with arbitrary input to the problem of sorting 0-1 inputs.

For any $a > 0$, let $h_a(x)$ be 1 if $x \ge a$; otherwise, 0. Given a sequence $I = (x_1, x_2, \ldots, x_N)$, let $h_a(I)$ be the 0-1 sequence $(h_a(x_1), h_a(x_2), \ldots, h_a(x_N))$. Let Q_i and $\widehat{Q}_i$ be the ith queue of ZZ-sort on input I and 0-1 input $h_a(I)$, respectively. In the initial distribution of I and $h_a(I)$, $h_a(x)$ is in $\widehat{Q}_i$ if and only if x is in Q_i for any $x \in I$. Given $Q_i = \{x_1, x_2, \ldots, x_{m_i}\}$, we define $h_a(Q_i) = \{h_a(x_1), h_a(x_2), \ldots, h_a(x_{m_i})\}$.

Lemma 1 *Suppose that ZZ-sort is executed on inputs I and $h_a(I)$ in parallel simultaneously. Then, after every iteration of the global phase, $\widehat{Q}_i = h_a(Q_i)$ for any $a > 0$.*

Proof: Without loss of generality, assume that N is a multiple of p, and p is even. Let $m = \frac{N}{p}$. We use induction on the number of iterations. Initially, the lemma holds by the distributions of I and $h_a(I)$. Assume that the lemma holds after k iterations. Suppose that $Q_i = \{y_{i,1}, y_{i,2}, \ldots, y_{i,m}\}$. Then, by induction hypothesis, $\widehat{Q}_i = h_a(Q_i) = \{h_a(y_{i,1}), h_a(y_{i,2}), \ldots, h_a(y_{i,m})\}$. We may assume that $y_{i,1} = \min_{1 \le j \le m}\{y_{i,j}\}$ and $y_{i,m} = \max_{1 \le j \le m}\{y_{i,j}\}$ for $0 \le i \le p-1$. Then $Y = \{y_{0,m}, y_{1,1}, y_{2,m}, y_{3,1}, \ldots, y_{p-2,m}, y_{p-1,1}\}$ is the set of elements selected from Q_i's in Step 1 of the $(k+1)$-th iteration. Let $Z = (z_0, z_1, \ldots, z_{p-1})$ be the sorted sequence of Y. For any $a > 0$, $h_a(Y)$ is the set of elements selected from $\widehat{Q}_i$'s

in Step 1 of the $(k+1)$-th iteration, and $h_a(Z)$ is the sorted sequence of $h_a(Y)$. Hence, after this step,

$$Q_i = \begin{cases} \{z_i, y_{i,1}, y_{i,2}, \ldots, y_{i,m-1}\} & \text{if } i \text{ is even} \\ \{z_i, y_{i,2}, y_{i,3}, \ldots, y_{i,m}\} & \text{if } i \text{ is odd} \end{cases}$$

and

$$\widehat{Q}_i = \begin{cases} \{h_a(z_i), h_a(y_{i,1}), h_a(y_{i,2}), \ldots, h_a(y_{i,m-1})\} & \text{if } i \text{ is even} \\ \{h_a(z_i), h_a(y_{i,2}), h_a(y_{i,3}), \ldots, h_a(y_{i,m})\} & \text{if } i \text{ is odd.} \end{cases}$$

Now, let $Q_i = \{y'_{i,1}, y'_{i,2}, \ldots, y'_{i,m}\}$, and assume that $y'_{i,1} = \min_{1\le j\le m}\{y'_{i,j}\}$ and $y'_{i,m} = \max_{1\le j\le m}\{y'_{i,j}\}$ for $0 \le i \le p-1$. Clearly, $Y' = \{y'_{0,1}, y'_{1,m}, y'_{2,1}, y'_{3,m}, \ldots, y'_{p-2,1}, y'_{p-1,m}\}$ is the set of elements selected from Q_i's in Step 2 of the $(k+1)$-th iteration. Let $Z' = (z'_0, z'_1, \ldots, z'_{p-1})$ be the sorted sequence of Y'. Then, $h_a(Y')$ is the set of elements selected from $\widehat{Q}_i$'s in Step 2 of the $(k+1)$-th iteration, and $h_a(Z')$ is the sorted sequence of $h_a(Y')$. After this step,

$$Q_i = \begin{cases} \{z'_i, y'_{i,2}, y'_{i,3}, \ldots, y'_{i,m}\} & \text{if } i \text{ is even} \\ \{z'_i, y'_{i,1}, y'_{i,2}, \ldots, y'_{i,m-1}\} & \text{if } i \text{ is odd} \end{cases}$$

and

$$\widehat{Q}_i = \begin{cases} \{h_a(z'_i), h_a(y'_{i,2}), h_a(y'_{i,3}), \ldots, h_a(y'_{i,m})\} & \text{if } i \text{ is even} \\ \{h_a(z'_i), h_a(y'_{i,1}), h_a(y'_{i,2}), \ldots, h_a(y'_{i,m-1})\} & \text{if } i \text{ is odd.} \end{cases}$$

The lemma holds after $k+1$ iterations. The induction is complete. □

Lemma 2 [Zero-One Principle] *If ZZ-sort globally sorts all* 0-1 *sequences of size N within t iterations, then it globally sorts any input sequence of size N within t iterations.*

Proof: Suppose to the contrary that ZZ-sort does not sort an input I in t iterations. Then, after t iterations, there must exist two queues Q_i and Q_j and $x \in Q_i$ and $y \in Q_j$ such that $i < j$ but $x > y$. Let $a = (x+y)/2$. Then $h_a(x) = 1$ and $h_a(y) = 0$. On the 0-1 input $h_a(I)$, by Lemma 1, after t iterations, $h_a(x) = 1 \in \widehat{Q}_i$ and $h_a(y) = 0 \in \widehat{Q}_j$ with $i < j$. This contradicts the assumption that ZZ-sort sorts any 0-1 input sequence in t iterations. □

Note that our use of the zero-one principle differs from its classical usage of proving correctness of a sorting network [9]. We apply this principle to prove the number of iterations required by an algorithm to obtain a correct global sorting result.

Theorem 1 *ZZ-sort sorts its p queues globally in* $\lceil \frac{N}{p} \rceil (\lceil \log_{\frac{4}{3}} p \rceil + 1) + 1$ *iterations on any input sequence of size N.*

Proof: By Lemma 2, we only need to show that ZZ-sort sorts the queues globally on any 0-1 input sequence of size N in $\lceil \frac{N}{p} \rceil (\lceil \log_{\frac{4}{3}} p \rceil + 1) + 1$ iterations. We first note the following simple and important fact: at any time during the global sorting phase, if the elements in the first k queues Q_i, $0 \le i \le k-1$, are all 0's, then for each subsequent step, these k queues contribute k 0's and receive k 0's so that each of these k queues remains to have all 0's.

Let I be an arbitrary 0-1 input sequence of size N. Let Z be the number of 0's in I. Assume that $0 < Z < N$; otherwise, the input is already sorted. Let $m = \lceil \frac{N}{p} \rceil$ and $k < \lceil \frac{Z}{m} \rceil$ for some $0 \le k \le p-1$. Consider the following two cases.

Case 1: $k = 1$. In this case, there are at most m 0's, and all 0's should go to Q_0. Suppose there exists a 0 not in Q_0. There are two subcases: (1) Q_0 initially contains at least one 0. Let $j \ge 1$ be the smallest integer such that in Step 1 of the jth iteration Q_0 does not receive one more 0. If there still exists 0's that have not been moved into Q_0, then after Step 2 of the the jth iteration, Q_1 has at least one 0. Consider subsequent iterations. In the $(j+c)$-th iteration, Q_0 receives one more 0 from Q_1 in the first step, and Q_1 is guaranteed to have at least one 0 after Step 2 of the iteration. This process continues until Q_0 receives all 0's. Since there are at most $m-1$ 0's initially not in Q_0, the total number of iterations required to move all 0's into Q_0 is m. (2) Q_0 initially contains all 1's. Then, after the first iteration, Q_0 receives at least one 0, and the rest of the queues have at most $m-1$ 0's. By the argument for (1), at most m more iterations are needed. Therefore, if $k = 1$, the global phase of ZZ-sort requires at most $m+1$ iterations.

Case 2: $k > 1$. In this case, there are at least $m(k-1)+1$ 0's. Since each queue has at most m elements, at least k queues contain 0's. After the first iteration, each of Q_i's such that $0 \le i \le \lceil \frac{k}{2} \rceil - 1$ contains at least one 0. This is because at least one of the two steps in an iteration will select at least $\lceil \frac{k}{2} \rceil$ 0's. For each iteration that follows the first iteration, at least $\lfloor \frac{k}{4} \rfloor$ 0's will be selected as minimums in each step. Consider the changes in the first $\lfloor \frac{k}{4} \rfloor$ queues after one iteration. For $0 \le i \le \lfloor \frac{k}{4} \rfloor - 1$, if Q_i contains some 1's before this iteration, then the number of 0's in queue Q_i will increase by at least one after this iteration; otherwise, no changes occur in Q_i. Hence, after m iterations, the first $\lfloor \frac{k}{4} \rfloor$ queues will contain all 0's. During each step of the later iterations, each of the first $\lfloor \frac{k}{4} \rfloor$ queues contributes one 0 and also receives one 0, thus incurs no changes. So we may consider the first $\lfloor \frac{k}{4} \rfloor$ queues as being "dropped out of the process." The number Z' of 0's in the remaining queues is at most $mk - m\lfloor \frac{k}{4} \rfloor \le m\lceil \frac{3k}{4} \rceil$. Let $k' = \lceil \frac{Z'}{m} \rceil \le \lceil \frac{3k}{4} \rceil$. The factor k is dropped by one-fourth after every m iterations. Hence, there can be at most $m\lceil \log_{\frac{4}{3}} k \rceil$ iterations in Case 2 before reaching to Case 1. After at most $m+1$ additional iterations, all 0's are in $Q_0, Q_2, \ldots, Q_k$, and all elements in each of $Q_0, Q_2, \ldots, Q_{k-1}$ are 0's. This distribution of 0's means that the queues are globally sorted.

Hence, after at most $m\lceil \log_{\frac{4}{3}} k \rceil + m + 1 \le \lceil \frac{N}{p} \rceil (\lceil \log_{\frac{4}{3}} p \rceil + 1) + 1$ iterations, the queues are globally sorted. Notice that the above proof works when m is the maxi-

mum of the queue sizes, even if the input elements are distributed unevenly among the queues. □

Theorem 2 *There exists an input instance of size N such that ZZ-sort requires* $\Omega(\frac{N}{p}\log p)$ *iterations in the global phase.*

Proof: Let $\frac{N}{p} = m$, and consider the input instance that each Q_i, $0 \le i \le p-1$ contains exactly $\frac{m}{3}$ 1's and $\frac{2m}{3}$ 0's. After first $\frac{m}{3}$ iterations, the first $\frac{p}{2}$ queues contain all 0's, thus dropping out of the process from now on, and each of the second $\frac{p}{2}$ queues is in the *reverse* state of having exactly $\frac{m}{3}$ 0's and $\frac{2m}{3}$ 1's. After another $\frac{m}{3}$ iterations, the second half of the remaining queues contain all 1's, thus dropping out of the process, and the first half return to their previous state of having $\frac{m}{3}$ 1's and $\frac{2m}{3}$ 0's. Thus each round of $\frac{m}{3}$ iterations eliminates exactly a half of the remaining queues. Hence, $\frac{m}{3}\log_2 p$ iterations are required to sort the given input. □

Theorem 2 indicates that the upper bound $O(\frac{N}{p}\log p)$ on the number of iterations in the global phase of ZZ-sort cannot be improved in the worst case, i.e. the number of global iterations is $\Theta(\frac{N}{p}\log p)$.

The "min-max heap" data structure in [2] can be used to maintain the elements of a queue as a priority queue, permitting operations *deletemin*, *deletemax* and *insert* to be performed in logarithmic time. The creation of the min-max heap, and the delete and insert operations can be done in-place, i.e. no extra memory space is required other than the space for storing the input elements. Final output is obtained by performing a sequence of parallel *deletemin* operations. Therefore, ZZ-sort provides a general scheme for in-place parallel sorting on a distributed memory parallel computer shown in Figure 1.

Theorem 3 *Given a distributed memory parallel computer C, ZZ-sort sorts N elements in-place in* $O(\frac{N}{p}\log p(f(p) + \log\frac{N}{p}))$ *time, where p is the number of processor/memory pairs used, and* $O(f(p))$ *is the time required for C to sort p elements, one element per processor.*

3. Implementation of ZZ-Sort on a Parallel Computer

We have implemented ZZ-sort on the MasPar MP-1 machine. MasPar MP-1 is an SIMD general purpose massively parallel computer. It has a data parallel unit (DPU) and a front end. The front end (e.g. a DECstation 5000) allows the users to access the DPU. The DPU consists of two parts: the array control unit (ACU), and the processing element (PE) array which has up to 16,384 PEs; each has a local memory of 64K bytes. The ACU controls the PEs, and has a memory which is shared by all PEs. The PEs operate synchronously executing instructions from the ACU. These PEs are arranged in a two-dimensional array with each PE connected to its 8 neighbors (toroidal wrap). This interconnection network is called the XNET in the MasPar MP-1 documentation. Interprocessor communications are carried out via

three mechanisms. ACU-PE array communication, XNET nearest-neighbor communication, and global crossbar router communication. The first mechanism supports (data and instruction) broadcast and global reduction operations. The other two support communications between all PEs. Massively parallel I/O is implemented in MasPar. The PE array communicates with a parallel disk array through the high-speed I/O subsystem; the basis of this subsystem is the global router, which is a multistage crossbar network. The MasPar MP-1 system has a programming language called MPL (MasPar Programming Language), which is an extension of C. MPL supports plural data types and plural operations. The plural operations are performed on plural variables, which are located in the local memories, in an SIMD manner. Plural **for** and **while** control statements are used to implement SIMD control flow for the distributed data.

In our implementation, each PE maintains a min-max heap stored in its local memory. Basic operations on heaps are insertion and deletion. The global phase is implemented in the form of a loop with a plural control variable. Within each iteration, extracted elements are sorted by the *psort* routine provided by the MPL. The loop is exited when the elements are detected globally sorted. The termination condition is evaluated as follows. Each PE accesses the minimum element of its (logical) next PE using the global router construct, and compares it to its own maximum element. Depending on the outcome of this comparison, the PE either set or reset a flag. The flag values in all PEs are ORed using the *reduceOR* function supported by ACU. The value returned, which is broadcast to all PEs, is used to determine whether or not further iterations are needed. Upon exit of the loop, each PE repeatedly deletes its minimal elements from its heap to obtain a locally sorted subsequence.

Our algorithm is executed on a MasPar MP-1 with 8192 processors, i.e. in our implementation, $p = 8192$. The number of iterations and the running time were the parameters tested. The input queue sizes were chosen $n = \frac{N}{p} = 250$, 500, 1000, and 2000 32-bit integers per processor, which correspond to $N = 2048000$, 4096000, 8192000, and 16384000 total elements, respectively. For each case, 20 runs of randomly generated inputs were carried out, and the results were averaged. A function of the system time was used to seed the random number generator for each run to ensure sufficient randomness across different runs. Our first experiment was on the number of iterations, and the results are listed in Table 1. Based on these results, we determined a lower bound $5.9m$ for the number of iterations by averaging the third column of Table 1. This shows that the average time complexity of our algorithm is $O(\frac{N}{p}(f(p) + \log \frac{N}{p}))$ for $\frac{N}{p} \leq 2000$. For each subsequent test, the termination con-

Table 1. Average number of iterations of ZZ-sort over 20 runs of random inputs.

Queue Size $m = \frac{N}{p}$	Number i of Iterations	$\frac{i}{m}$
250	1495.25	4.98
500	3097.31	6.19
1000	6303.50	6.30
2000	12919.50	6.46

Table 2. Average execution time of ZZ-sort on MasPar over 20 runs of random inputs.

Queue Size $m = \frac{N}{p}$	Time in Milliseconds
250	40771
500	91728
1000	203175
2000	445908

dition was evaluated after $5.9m$ iterations, at the end of every additional iteration. This rule was used to improve the algorithm. The test results were based on this improved implementation of the algorithm. Table 2 shows the running time for each queue size.

Let us consider the speedup and efficiency of ZZ-sort on MasPar MP-1. Since we have 8192 PEs, to make a reasonable evaluation, we have to run an optimal sequential sorting algorithm on a single PE of the MasPar MP-1 machine with $N = 8192 \times m$ integers, where m is sufficiently large. Unfortunately, the local memory associated with each PE in MasPar MP-1 has 64K bytes so that the largest n is 2 (each integer requires 4 bytes). We have to come up with an estimation on the time required to sort sufficiently large N number of integers on a single PE of MasPar MP-1. We do this by experiment. The running time is proportional to the number of executed instructions. For an implementation of an algorithm, the number of executed instructions depends on the complexity of the algorithm and the code generated by the compiler. We know that heap sort is an optimal-time algorithm, and both of its worst case and average case complexities are $O(N \log N)$. The average running times T of heap sort we obtained for $N = 500$ and $N = 5000$ on a single MasPar PE are listed in Table 3. The third column of Table 3 shows the time $\frac{T}{N \log_2 N}$ per *logical operation*. The total number of logical operations, each consists of no more than a constant number of machine instructions, is proportional to the number of instructions (generated by the MasPar MPL compiler) executed. By averaging the $\frac{T}{N \log_2 N}$ ratio, we determine that 1.858 milliseconds per logical operation. We use $1.858 \times N \log_2 N$ to estimate the time required for sorting N integers on a single PE of MasPar MP-1. For example, for $N = 8192 \times 250 = 2048000$, this time is estimated to be 79908864 milliseconds. From Table 2, the time taken by 8192 processors to sort the same number of integers is 40771 milliseconds. Then, for $N = 2048000$ and $p = 8192$ we obtain an average speedup of 1960 on the MasPar MP-1 machine; the average efficiency is 23.9%. In Table 4 we list the average speedup and efficiency results based on our experiments. These results are by no

Table 3. Average execution time of heap sort on single MasPar PE over 20 runs of random input of size N.

Input Size N	Time T in Milliseconds	$\frac{T}{N \log N}$
500	2481	1.83
5000	34793	1.88

Table 4. Average speedup and efficiency of ZZ-sort over 20 runs of random inputs.

Input Size N	Queue Size $m = \frac{N}{p}$	Average Speedup	Average Efficiency
2048000	250	1960	23.9%
4096000	500	1825	22.3%
8192000	1000	1723	21.0%
16384000	2000	1638	20.0%

means to be precise; they are rather used to approximate the real speedup and efficiency performances. From experiments, we see that the speedup and efficiency of our ZZ-sort implementation are not so sensitive to the input size. As the input size increased 800% to 16.384 millions elements, the speedup and efficiency decreased by approximately 16.4%. This indicates that ZZ-sort is quite scalable.

The in-place bitonic sort implemented on CM-2 has time complexity $O(\frac{N}{p}\log^2 N)$ [3]. No speedup and efficiency data of this implementation were reported in [3]. Since the performance of the two machines, CM-2 and MasPar, are different, we compare the actual performances of our ZZ-sort and the bitonic sort of [3] by estimation. The ZZ-sort implemented on CM-2, using the generic bitonic sort (one number per processor) as SORT, would have time complexity $O(\frac{N}{p}\log p(f(p)+\log\frac{N}{p})) = O(\frac{N}{p}\log^3 p + \frac{N}{p}\log p\log\frac{N}{p})$. Table 1 indicates that for random inputs with $\frac{N}{p} \le 2000$, ZZ-sort actually takes $O(\frac{N}{p}(\log^2 p + \log\frac{N}{p}))$ time. Thus, both our ZZ-sort and the bitonic sort of [3] have speedups of the same order $O(\frac{p}{\log N})$ for practical $\frac{N}{p}$ ratios. Therefore, the performance of ZZ-sort, as shown in Table 4, is acceptable.

4. Final Remarks

We presented a general parallel sorting scheme, ZZ-sort, that can be used to derive a class of efficient in-place parallel sorting algorithms. We discussed a particular implementation of such an algorithm on a MasPar parallel computer. We conclude this paper by showing an application of our ZZ-sort algorithm on a potential future parallel computer architecture.

In recent years, parallel computation models based on processor arrays supplemented with buses have drawn considerable attention. A bus system whose configuration can be dynamically altered is called a reconfigurable bus system. Several processor arrays equipped with reconfigurable buses have been proposed, and efficient parallel algorithms have been designed for these architectures (e.g. [8, 10, 11, 12, 13, 14, 15]). The most important feature of this class of architectures is that dynamically reconfigurable communication paths are used to perform tasks that can be done only by processors in other parallel architectures. It is assumed that broadcasting and long-distance communications on a bus can be completed in constant time. The advances of optical technologies have made these assumptions for an optical bus connecting many processors more and more acceptable. There

are many desirable characteristics of optical interconnections: high speed and bandwidth, increased fanout, high reliability, supporting longer interconnection lengths, low power requirements, etc. Currently, several projects are being conducted to realize massively parallel computing using optical interconnections.

Let A be a parallel algorithm solving a problem Π of input size p on a $p \times p$ reconfigurable mesh M. We say that A is an *adaptive algorithm* if it can efficiently solve problem Π of any input size, assuming sufficient memory space is available. The sorting algorithms given in [8, 13, 11, 14] sort p elements on a $p \times p$ reconfigurable mesh in $O(1)$ time. These algorithms are not adaptive, because the clever bus control techniques used to reconfigure communication paths as an integral part of computation cannot be used to sort more than p elements. Our ZZ-sort can be used to convert these sorting algorithms into adaptive ones. For our purpose, the $p \times p$ reconfigurable mesh is used as a sorting network. As in existing reconfigurable mesh sorting algorithms, the p processors on one boundary side of the mesh, which are referred to as input processors, provide input elements (each processor provides one element). If we allow each input processor to have a local memory to implement a min-max heap, then we have an implementation of ZZ-sort using the entire mesh as the SORT procedure. Clearly, this version of ZZ-sort is adaptive and in-place. Based on Theorem 2, we have the following result: using a $p \times p$ reconfigurable mesh, ZZ-sort sorts N numbers in-place in $O(\frac{N}{p} \log p \log \frac{N}{p})$ time. The termination condition of ZZ-sort on a $p \times p$ reconfigurable mesh can be tested by using the $O(1)$-time prefix-sums algorithm so that the global phase of the algorithm can be terminated as soon as the queues are globally sorted.

Acknowledgments

This work was partly supported by NSF grant ECS-9626215 and Louisiana grant LEQSF(1996-99)-RD-A-16.

References

1. H. R. Arabnia and S. M. Bhandarkar, Parallel Stereocorrelation on a Reconfigurable Multi-Ring Network, *The Journal of Supercomputing*, **10**, pp. 243–269, 1996.
2. M. D. Atkinson, J. R. Sack, N. Santoro, and T. Strothotte, Min-max Heaps and Generalized Priority-Queues, *Communications of ACM*, **29**, pp. 996-1000, 1986.
3. G. E. Blelloch, C. E. Leiserson, B. M. Maggs, C. G. Plaxton, S. J. Smith, and M. Zagha, A Comparison of Sorting Algorithms for the Connection Machine CM-2, to appear in *Communication of ACM*.
4. G. M. Amdahl, Validity of Single-Processor Approach to Achieving Large-Scale Computing Capability, *Proc. AFIPS Conference*, pp. 483–485, 1967.
5. X. Guan and M. A. Langston, Time-Space Optimal Parallel Sorting and Merging, *IEEE Transactions on Computers*, **40**, pp. 596-602, 1991.
6. J. Gustafson, Reevaluating Amdahl's Law, *Communication of ACM*, **31**, pp. 532–533, 1988.
7. K. Hwang, *Advanced Computer Architecture: Parallelism, Scalability, Programmability*, McGraw-Hill, 1993.
8. J. Jang and V. K. Prasanna, An Optimal Sorting Algorithm on Reconfigurable Mesh, *Proc. 6th Int. Parallel Processing Symposium*, pp. 130–137, 1992.

9. D. E. Knuth, *The Art of Computer Programming*, **Vol. 3**, Addison Wesley, 1973.
10. H. Li and M. Maresca, Polymorphic-Torus Network, *IEEE Transactions on Computers*, **38**, pp. 1345–1351, 1989.
11. R. Lin, S. Olariu, J. Schwing, and J. Zhang, Sorting in O(1) Time on a Reconfigurable Mesh of Size $n \times n$, *Parallel Computing: From Theory to Sound Practice, Proceedings of EWPC'92*, pp. 16–27, 1992.
12. R. Miller, V. K. Prasanna-Kumar, D. Reisis, and Q. F. Stout, Meshes with Reconfigurable Buses, *Proceedings of the International Conference on Parallel Processing*, **Vol. 1**, pp. 205–208, 1988.
13. M. Nigam and Sahni, Sorting n Numbers on $n \times n$ Mesh with Buses, Technical Report #92-5, University of Florida, Gainsville, 1992.
14. S. Olariu and J. Schwing, A New Deterministic Sampling Scheme with Applications to Broadcast Efficient Sorting on the Reconfigurable Mesh, *Journal of Parallel and Distributed Computing*, to appear.
15. S. Olariu, J. Schwing, and J. Zhang, Applications of Reconfigurable Meshes to Constant-Time Computations, *Parallel Computing*, **19**, pp. 229–237, 1993.
16. X. H. Sun and L. M. Ni, Scalable Problems and Memory-Bounded Speedup, *Journal of Parallel and Distributed Computing*, **19**, pp. 27–37, 1993.
17. Y. Zhang and S. Q. Zheng, Design and Analysis of a Systolic Sorting Architecture, *Proceedings of 7th Symposium on Parallel and Distributed Processing*, pp. 652–659, 1995.